How to convert a string to lower case in Bash?

Asked 11 years, 10 months ago Active 18 days ago Viewed 1.1m times

1527

365

Is there a way in bash to convert a string into a lower case string?

For example, if I have:

```
a="Hi all"
```

I want to convert it to:

```
"hi all"
```

string bash shell lowercase

Share Follow

edited Mar 29 '20 at 14:36
thor
20.1k 26 78 157

asked Feb 15 '10 at 7:02
assassin
17.6k 10 27 42

1 **See also**: stackoverflow.com/questions/11392189 – dreftymac Mar 29 '18 at 16:04

25 Answers

Active | Oldest | Votes

2607

The are various ways:

POSIX standard

tr

```
$ echo "$a" | tr '[:upper:]' '[:lower:]'
hi all
```


Non-POSIX

You may run into portability issues with the following examples:

Bash 4.0

```
$ echo "${a,,}"
hi all
```

sed

```
$ echo "$a" | sed -e 's/\(.*\)/\L\1/'
hi all
# this also works:
$ sed -e 's/\(.*\)/\L\1/' <<< "$a"
hi all
```

Perl

```
$ echo "$a" | perl -ne 'print lc'
hi all
```

Bash

```
lc(){
    case "$1" in
        [A-Z])
        n=$(printf "%d" "'$1")
        n=$((n+32))
        printf \\$(printf "%o" "$n")
        ;;
        *)
        printf "%s" "$1"
        ;;
    esac
}
word="I Love Bash"
for((i=0;i<${#word};i++))
do
    ch="${word:$i:1}"
    lc "$ch"
done
```

Note: YMMV on this one. Doesn't work for me (GNU bash version 4.2.46 and 4.0.33 (and same behaviour 2.05b.0 but nocasematch is not implemented)) even with using `shopt -u nocasematch`;

 **3,647** 1 26 33  **299k** 52 247 336

10 Am I missing something, or does your last example (in Bash) actually do something completely different? It works for "ABX", but if you instead make `word="Hi All"` like the other examples, it returns `ha`, not `hi all`. It only works for the capitalized letters and skips the already-lowercased letters. – jangosteve Jan 14 '12 at 21:58

28 Note that only the `tr` and `awk` examples are specified in the POSIX standard. – Richard Hansen Feb 3 '12 at 18:55

189 `tr '[:upper:]' '[:lower:]'` will use the current locale to determine uppercase/lowercase equivalents, so it'll work with locales that use letters with diacritical marks. – Richard Hansen Feb 3 '12 at 18:58

13 How does one get the output into a new variable? Ie say I want the lowercased string into a new variable? – Adam Parkin Sep 25 '12 at 18:01

66 @Adam: `b="$(echo $a | tr '[A-Z]' '[a-z]')"` – Tino Nov 14 '12 at 15:39

495

In Bash 4:

To lowercase

```
$ string="A FEW WORDS"
$ echo "${string,}"
a FEW WORDS
$ echo "${string,,}"
a few words
$ echo "${string,,[AEIUO]}"
a FeW WoRDS

$ string="A Few Words"
$ declare -l string
$ string=$string; echo "$string"
a few words
```

To uppercase

```
$ string="a few words"
$ echo "${string^}"
A few words
$ echo "${string^^}"
A FEW WORDS
$ echo "${string^^[aeiou]}"
```

```
$ string="A Few Words"
$ echo "${string~~}"
a fEW wORDS
$ string="A FEW WORDS"
$ echo "${string~}"
a FEW WORDS
$ string="a few words"
$ echo "${string~}"
A few words
```

Capitalize (undocumented, but optionally configurable at compile time)

```
$ string="a few words"
$ declare -c string
$ string=$string
$ echo "$string"
A few words
```

Title case:

```
$ string="a few words"
$ string=($string)
$ string="${string[@]^}"
$ echo "$string"
A Few Words

$ declare -c string
$ string=(a few words)
$ echo "${string[@]}"
A Few Words

$ string="a FeW WOrdS"
$ string=${string,,}
$ string=${string~}
$ echo "$string"
A few words
```

To turn off a `declare` attribute, use `+`. For example, `declare +c string`. This affects subsequent assignments and not the current value.

The `declare` options change the attribute of the variable, but not the contents. The reassignments in my examples update the contents to show the changes.

Edit:

 **36.2k** 16 89 94  **316k** 87 361 423

5 Quite bizzare, "^^" and ",," operators don't work on non-ASCII characters but "~~" does... So `string="łódź"; echo ${string~~}` will return "ŁÓDŹ", but `echo ${string^^}` returns "łóDź". Even in `LC_ALL=pl_PL.utf-8` . That's using bash 4.2.24. – Hubert Kario Jul 12 '12 at 16:48

2 @HubertKario: That's weird. It's the same for me in Bash 4.0.33 with the same string in `en_US.UTF-8` . It's a bug and I've reported it. – Dennis Williamson Jul 12 '12 at 18:20

1 @HubertKario: Try `echo "$string" | tr '[:lower:]' '[:upper:]'` . It will probably exhibit the same failure. So the problem is at least partly not Bash's. – Dennis Williamson Jul 13 '12 at 0:44

1 @DennisWilliamson: Yeah, I've noticed that too (see comment to Shuvalov answer). I'd just say, "this stuff is only for ASCII", but then it's the "~~" operator that does work, so it's not like the code and translation tables aren't already there... – Hubert Kario Jul 14 '12 at 14:13

4 @HubertKario: The Bash maintainer has acknowledged the bug and stated that it will be fixed in the next release. – Dennis Williamson Jul 14 '12 at 14:27

135

```
echo "Hi All" | tr "[:upper:]" "[:lower:]"
```

Share Follow

edited May 7 '13 at 14:46  Steven Penny **1**

answered Feb 15 '10 at 7:13  shuvalov **4,225** 2 17 17

4 @RichardHansen: `tr` doesn't work for me for non-ACII characters. I do have correct locale set and locale files generated. Have any idea what could I be doing wrong? – Hubert Kario Jul 12 '12 at 16:56

FYI: This worked on Windows/Msys. Some of the other suggestions did not. – kodybrown Oct 23 '14 at 16:42

7 Why is `[:upper:]` needed? – mgutt Aug 8 '19 at 22:47

The same question why `[:upper:]` is needed. – MaXi32 Jul 17 at 14:16

93

tr:

```
a="$(tr [A-Z] [a-z] <<< "$a")"
```



```
y/ABCDEFGHIJKLMNOPQRSTUVWXYZ/abcdefghijklmnopqrstuvwxyz/
```

Share Follow

edited Jun 26 '13 at 17:19

Peter Mortensen
29.3k 21 97 124

answered Feb 15 '10 at 7:03

Ignacio Vazquez-Abrams
724k 144 1276 1310

2 `+1 a="$(tr [A-Z] [a-z] <<< "$a")"` looks easiest to me. I am still a beginner... – Sandeepan Nath Feb 2 '11 at 11:12

2 I strongly recommend the `sed` solution; I've been working in an environment that for some reason doesn't have `tr` but I've yet to find a system without `sed`, plus a lot of the time I want to do this I've just done something else in `sed` anyway so can chain the commands together into a single (long) statement. – Haravikk Oct 19 '13 at 12:54

2 The bracket expressions should be quoted. In `tr [A-Z] [a-z] A`, the shell may perform filename expansion if there are filenames consisting of a single letter or *nullglob* is set. `tr "[A-Z]" "[a-z]" A` will behave properly. – Dennis Nov 6 '13 at 19:49

3 @CamiloMartin it's a BusyBox system where I'm having that problem, specifically Synology NASes, but I've encountered it on a few other systems too. I've been doing a lot of cross-platform shell scripting lately, and with the requirement that nothing extra be installed it makes things very tricky! However I've yet to encounter a system without `sed` – Haravikk Jun 15 '14 at 10:51

3 Note that `tr [A-Z] [a-z]` is incorrect in almost all locales. for example, in the `en-US` locale, `A-Z` is actually the interval `AaBbCcDdEeFfGgHh...XxYyZ`. – fuz Jan 31 '16 at 14:54

67

I know this is an oldish post but I made this answer for another site so I thought I'd post it up here:

UPPER -> lower: use python:

```
b=`echo "print '$a'.lower()" | python`
```

Or Ruby:

```
b=`echo "print '$a'.downcase" | ruby`
```

Or Perl:

Or Awk:

```
b=`echo "$a" | awk '{ print tolower($1) }'`
```

Or Sed:

```
b=`echo "$a" | sed 's/./\L&/g'`
```

Or Bash 4:

```
b=${a,,}
```

Or NodeJS:

```
b=`node -p "\"$a\".toLowerCase()"`
```

You could also use `dd` :

```
b=`echo "$a" | dd  conv=lcase 2> /dev/null`
```

lower -> UPPER:

use python:

```
b=`echo "print '$a'.upper()" | python`
```

Or Ruby:

```
b=`echo "print '$a'.upcase" | ruby`
```

Or Perl:

```
b=`perl -e "print uc('$a');"`
```



```
b=`echo "$a" | awk '{ print toupper($1) }'`
```

Or Sed:

```
b=`echo "$a" | sed 's/./\U&/g'`
```

Or Bash 4:

```
b=${a^^}
```

Or NodeJS:

```
b=`node -p "\"$a\".toUpperCase()"`
```

You could also use `dd`:

```
b=`echo "$a" | dd  conv=ucase 2> /dev/null`
```

Also when you say 'shell' I'm assuming you mean `bash` but if you can use `zsh` it's as easy as

```
b=$a:l
```

for lower case and

```
b=$a:u
```

for upper case.

Share Follow

edited Dec 4 '20 at 10:47

answered May 14 '14 at 9:36

nettux
4,704 2 21 32

@JESii both work for me upper -> lower and lower-> upper. I'm using sed 4.2.2 and Bash 4.3.42(1) on 64bit Debian Stretch. – nettux Nov 20 '15 at 14:33

1 Hi, @nettux443... I just tried the bash operation again and it still fails for me with the error message "bad substitution". I'm on OSX using homebrew's bash: GNU bash, version 4.3.42(1)-release (x86_64-apple-

I prefer using the dd solution. Please note that you need to be root to get it working – lnetphantom Mar 6

'19 at 14:05

31

In zsh:

```
echo $a:u
```

Gotta love zsh!

Share Follow

edited Jan 27 '11 at 6:03

BenV
11.5k 10 57 89

answered Jan 27 '11 at 5:37

Scott Smedley
1,609 18 27

4 or $a:l for lower case conversion – Scott Smedley Jan 27 '11 at 5:39

1 Add one more case: `echo ${(C)a} #Upcase the first char only` – biocyberman Jul 24 '15 at 23:26

31

Bash 5.1 provides a straight forward way to do this with the `L` parameter transformation:

```
${var@L}
```

So for example you can say:

```
$ v="heLLo"
$ echo "${v@L}"
hello
```

You can also do uppercase with `U`:

```
$ v="hello"
$ echo "${v@U}"
HELLO
```

And uppercase the first letter with `u`:

▼▼ harming

242k 89 491 548

1 Absolutely deserves more upvotes than it currently has! – Umlin Jan 11 at 9:04

1 @Umlin it is a brand new feature, so it is normal that didn't get much attention yet. – fedorqui 'SO stop harming' Jan 11 at 9:26

I can't use this yet, but glad to know it's a thing! – Lee Harrison Oct 4 at 19:17

@LeeHarrison you can install it very easily, I recommend it to you :) – fedorqui 'SO stop harming' Oct 4 at 19:22

19

Using GNU `sed` :

```
sed 's/.*/\L&/'
```

Example:

```
$ foo="Some STRIng";
$ foo=$(echo "$foo" | sed 's/.*/\L&/')
$ echo "$foo"
some string
```

Share Follow

edited Jan 16 '16 at 12:25
tripleee
150k 26 224 283

answered Sep 26 '13 at 15:45
devnull
108k 29 217 212

17

Pre Bash 4.0

Bash Lower the Case of a string and assign to variable

```
VARIABLE=$(echo "$VARIABLE" | tr '[:upper:]' '[:lower:]')

echo "$VARIABLE"
```

Share Follow

edited Jan 16 '16 at 12:26

community wiki
2 revs, 2 users 94%
hawkeye126

on my systems – Tino Jan 17 '16 at 14:28

12

You can try this

```
s="Hello World!"

echo $s  # Hello World!

a=${s,,}
echo $a  # hello world!

b=${s^^}
echo $b  # HELLO WORLD!
```

```
root@ubuntu:/home/tr:con# bash mysh.sh
Hello World!
hello world!
HELLO WORLD!
root@ubuntu:/home/tr:con#
```

ref : http://wiki.workassis.com/shell-script-convert-text-to-lowercase-and-uppercase/

Share Follow

answered Mar 23 '17 at 6:48

Bikesh M

7,411 5 36 51

great! Was about to make an answer like this. Many answers adding lots of unneeded info – yosefrow Feb 16 at 13:52

11

For a standard shell (without bashisms) using only builtins:

```
uppers=ABCDEFGHIJKLMNOPQRSTUVWXYZ
lowers=abcdefghijklmnopqrstuvwxyz

lc(){ #usage: lc "SOME STRING" -> "some string"
    i=0
    while ([ $i -lt ${#1} ]) do
        CUR=${1:$i:1}
        case $uppers in
            *$CUR*)CUR=${uppers%$CUR*};OUTPUT="${OUTPUT}${lowers:${#CUR}:1}";;
```

```
uc(){ #usage: uc "some string" -> "SOME STRING"
    i=0
    while ([ $i -lt ${#1} ]) do
        CUR=${1:$i:1}
        case $lowers in
            *$CUR*)CUR=${lowers%$CUR*};OUTPUT="${OUTPUT}${uppers:${#CUR}:1}";;
            *)OUTPUT="${OUTPUT}$CUR";;
        esac
        i=$((i+1))
    done
    echo "${OUTPUT}"
}
```

Share Follow

edited Jun 26 '13 at 17:21

Peter Mortensen
29.3k 21 97 124

answered Jan 21 '12 at 10:27

technosaurus
7,231 1 28 50

I wonder if you didn't let some bashism in this script, as it's not portable on FreeBSD sh: ${1:$...}: Bad substitution – Dereckson Nov 23 '14 at 19:52

2 Indeed; substrings with `${var:1:1}` are a Bashism. – tripleee Apr 14 '15 at 7:09

This approach has pretty bad performance metrics. See my answer for metrics. – Dejay Clayton Jul 28 '18 at 23:39

11

In bash 4 you can use typeset

Example:

```
A="HELLO WORLD"
typeset -l A=$A
```

Share Follow

edited Sep 26 '13 at 15:14

oHo
43.9k 26 148 186

answered Aug 21 '13 at 10:21


c4f4t0r
1,399 13 23

Ah, we poor macOS users, it's 2020 and Apple has dropped support for `bash` which is 'stuck' at 3.2.57(1)... (Note: aye, I'm aware we can always install a more recent `bash` from `homebrew` ...) – Gwyneth Llewelyn Sep 9 '20 at 16:37


Tomas Jablonskis
3,828 4 21 37


Atul23
121 1 5

It is better to use gawk which properly handles UTF8-encoded characters (and different languages charset). 'Awk tolower' will fail on something like "ЛШТШФУМ АЩЬФ". – Vit Jul 27 '20 at 21:59

the awk available on macOS 11.6 works flawlessly: `echo 'Đêm lưu trú năm nay' | awk '{ print tolower($0); }' => đêm lưu trú năm nay` , and `echo 'ЛШТШФУМ АЩЬФ' | awk '{ print tolower($0); }' => лштшфум ащьф` – Walter Tross Sep 29 at 15:02

Regular expression

7

I would like to take credit for the command I wish to share but the truth is I obtained it for my own use from http://commandlinefu.com. It has the advantage that if you `cd` to any directory within your own home folder that is it will change all files and folders to lower case recursively please use with caution. It is a brilliant command line fix and especially useful for those multitudes of albums you have stored on your drive.

```
find . -depth -exec rename 's/(.*)\/([^\/]*)/$1\/\L$2/' {} \;
```

You can specify a directory in place of the dot(.) after the find which denotes current directory or full path.

I hope this solution proves useful the one thing this command does not do is replace spaces with underscores - oh well another time perhaps.

Share Follow

edited Jan 27 '11 at 6:02

BenV
11.5k 10 57 89

answered Jan 24 '11 at 13:53

Derek Shaw
95 1 1

This didn't work for me for whatever reason, though it looks fine. I did get this to work as an alternative though: find . -exec /bin/bash -c 'mv {} `tr [A-Z] [a-z] <<< {}` \; – John Rix Jun 26 '13 at 15:58

This needs `prename` from `perl` : `dpkg -S "$(readlink -e /usr/bin/rename)"` gives `perl: /usr/bin/prename` – Tino Dec 11 '15 at 16:27

Converting case is done for alphabets only. So, this should work neatly.

```
cat path/to/file/filename | tr 'a-z' 'A-Z'
```

For converting from upper case to lower case

```
cat path/to/file/filename | tr 'A-Z' 'a-z'
```

For example,

filename:

```
my name is xyz
```

gets converted to:

```
MY NAME IS XYZ
```

Example 2:

```
echo "my name is 123 karthik" | tr 'a-z' 'A-Z'
# Output:
# MY NAME IS 123 KARTHIK
```

Example 3:

```
echo "my name is 123 &&^&& #@$#@%%& kAR2~thik" | tr 'a-z' 'A-Z'
# Output:
# MY NAME IS 123 &&^&& #@0@%%& KAR2~THIK
```

Share Follow

edited Mar 6 '18 at 12:21

answered Mar 6 '18 at 12:15

theBuzzyCoder
2,284 2 25 25

5

Many answers using external programs, which is not really using `Bash`.

If you know you will have Bash4 available you should really just use the `${VAR,,}` notation (it is

The way this works is by getting the ASCII integer representation of each char with `printf` and then `adding 32` if `upper-to->lower`, or `subtracting 32` if `lower-to->upper`. Then use `printf` again to convert the number back to a char. From `'A' -to-> 'a'` we have a difference of 32 chars.

Using `printf` to explain:

```
$ printf "%d\n" "'a"
97
$ printf "%d\n" "'A"
65


97 - 65 = 32
```

And this is the working version with examples.
Please note the comments in the code, as they explain a lot of stuff:

```
#!/bin/bash

# lowerupper.sh

# Prints the lowercase version of a char
lowercaseChar(){
    case "$1" in
        [A-Z])
            n=$(printf "%d" "'$1")
            n=$((n+32))
            printf \\$(printf "%o" "$n")
            ;;
        *)
            printf "%s" "$1"
            ;;
    esac
}

# Prints the lowercase version of a sequence of strings
lowercase() {
    word="$@"
    for((i=0;i<${#word};i++)); do
        ch="${word:$i:1}"
        lowercaseChar "$ch"
    done
}

# Prints the uppercase version of a char
uppercaseChar(){
    case "$1" in
        [a-z])
```

```
# Prints the uppercase version of a sequence of strings
uppercase() {
    word="$@"
    for((i=0;i<${#word};i++)); do
        ch="${word:$i:1}"
        uppercaseChar "$ch"
    done
}

# The functions will not add a new line, so use echo or
# append it if you want a new line after printing

# Printing stuff directly
lowercase "I AM the Walrus!"$'\n'
uppercase "I AM the Walrus!"$'\n'

echo "----------"

# Printing a var
str="A StRing WITH mixed sTUFF!"
lowercase "$str"$'\n'
uppercase "$str"$'\n'

echo "----------"

# Not quoting the var should also work,
# since we use "$@" inside the functions
lowercase $str$'\n'
uppercase $str$'\n'

echo "----------"

# Assigning to a var
myLowerVar="$(lowercase $str)"
myUpperVar="$(uppercase $str)"
echo "myLowerVar: $myLowerVar"
echo "myUpperVar: $myUpperVar"

echo "----------"

# You can even do stuff like
if [[ 'option 2' = "$(lowercase 'OPTION 2')" ]]; then
    echo "Fine! All the same!"
else
    echo "Ops! Not the same!"
fi

exit 0
```

And the results after running this:


```
a string with mixed stuff!
```

```
A STRING WITH MIXED STUFF!
----------
myLowerVar: a string with mixed stuff!
myUpperVar: A STRING WITH MIXED STUFF!
----------
Fine! All the same!
```

This should only work for ASCII characters though.

For me it is fine, since I know I will only pass ASCII chars to it.
I am using this for some case-insensitive CLI options, for example.

Share Follow

edited May 16 '17 at 11:22

answered May 16 '17 at 10:04

Gus Neves
430 6 6

4

If using v4, this is baked-in. If not, here is a **simple, widely applicable** solution. Other answers (and comments) on this thread were quite helpful in creating the code below.

```
# Like echo, but converts to lowercase
echolcase () {
    tr [:upper:] [:lower:] <<< "${*}"
}

# Takes one arg by reference (var name) and makes it lowercase
lcase () {
    eval "${1}"=\'$(echo ${!1//\'/"'\''"} | tr [:upper:] [:lower:] )\'
}
```

Notes:

- Doing: `a="Hi All"` and then: `lcase a` will do the same thing as: `a=$( echolcase "Hi All" )`
- In the lcase function, using `${!1//\'/"'\''"}` instead of `${!1}` allows this to work even when the string has quotes.

Share Follow

answered Mar 22 '13 at 22:42

Stephen M. Harris
6,255 2 36 44

I've compared my approach with several other answers on this page that are compatible with Bash 3.2. My approach is far more performant than most approaches documented here, and is even faster than `tr` in several cases.

Here are the timing results for 1,000 iterations of 25 characters:

- 0.46s for my approach to lowercase; 0.96s for uppercase
- 1.16s for Orwellophile's approach to lowercase; 1.59s for uppercase
- 3.67s for `tr` to lowercase; 3.81s for uppercase
- 11.12s for ghostdog74's approach to lowercase; 31.41s for uppercase
- 26.25s for technosaurus' approach to lowercase; 26.21s for uppercase
- 25.06s for JaredTS486's approach to lowercase; 27.04s for uppercase

Timing results for 1,000 iterations of 445 characters (consisting of the poem "The Robin" by Witter Bynner):

- 2s for my approach to lowercase; 12s for uppercase
- 4s for `tr` to lowercase; 4s for uppercase
- 20s for Orwellophile's approach to lowercase; 29s for uppercase
- 75s for ghostdog74's approach to lowercase; 669s for uppercase. It's interesting to note how dramatic the performance difference is between a test with predominant matches vs. a test with predominant misses
- 467s for technosaurus' approach to lowercase; 449s for uppercase
- 660s for JaredTS486's approach to lowercase; 660s for uppercase. It's interesting to note that this approach generated continuous page faults (memory swapping) in Bash

Solution:

```
#!/bin/bash
set -e
set -u

declare LCS="abcdefghijklmnopqrstuvwxyz"
declare UCS="ABCDEFGHIJKLMNOPQRSTUVWXYZ"

function lcase()
{
```

```
  done

  echo -n "${TARGET}"
}

function ucase()
{
  local TARGET="${1-}"
  local LCHAR=''
  local LOFFSET=''

  while [[ "${TARGET}" =~ ([a-z]) ]]
  do
    LCHAR="${BASH_REMATCH[1]}"
    LOFFSET="${LCS%%${LCHAR}*}"
    TARGET="${TARGET//${LCHAR}/${UCS:${#LOFFSET}:1}}"
  done

  echo -n "${TARGET}"
}
```

The approach is simple: while the input string has any remaining uppercase letters present, find the next one, and replace all instances of that letter with its lowercase variant. Repeat until all uppercase letters are replaced.

Some performance characteristics of my solution:

1. Uses only shell builtin utilities, which avoids the overhead of invoking external binary utilities in a new process
2. Avoids sub-shells, which incur performance penalties
3. Uses shell mechanisms that are compiled and optimized for performance, such as global string replacement within variables, variable suffix trimming, and regex searching and matching. These mechanisms are far faster than iterating manually through strings
4. Loops only the number of times required by the count of unique matching characters to be converted. For example, converting a string that has three different uppercase characters to lowercase requires only 3 loop iterations. For the preconfigured ASCII alphabet, the maximum number of loop iterations is 26
5. `UCS` and `LCS` can be augmented with additional characters

Share Follow

edited Jul 28 '18 at 23:45

answered Jul 28 '18 at 17:37

Dejay Clayton


```
{
```

```
    local __word=$1
    local __len=${#__word}
    local __char
    local __octal
    local __decimal
    local __result

    for (( i=0; i<__len; i++ ))
    do
       __char=${__word:$i:1}
       case "$__char" in
          [A-Z] )
             printf -v __decimal '%d' "'$__char"
             printf -v __octal '%03o' $(( $__decimal ^ 0x20 ))
             printf -v __char \\$__octal
             ;;
       esac
       __result+="$__char"
    done
    REPLY="$__result"
}
```

technosaurus's answer had potential too, although it did run properly for mee.

Share Follow

edited May 23 '17 at 11:55
Community Bot
1 1

answered Mar 24 '13 at 13:43
Orwellophile
11.8k 3 62 39

Not bad! For an analysis of the performance of this approach, please see my answer for metrics.
– Dejay Clayton Jul 28 '18 at 23:40

3

In spite of how old this question is and similar to this answer by technosaurus. I had a hard time finding a solution that was portable across most platforms (That I Use) as well as older versions of bash. I have also been frustrated with arrays, functions and use of prints, echos and temporary files to retrieve trivial variables. This works very well for me so far I thought I would share. My main testing environments are:

1. GNU bash, version 4.1.2(1)-release (x86_64-redhat-linux-gnu)
2. GNU bash, version 3.2.57(1)-release (sparc-sun-solaris2.10)

done

Simple C-style for loop to iterate through the strings. For the line below if you have not seen anything like this before this is where I learned this. In this case the line checks if the char ${input:$i:1} (lower case) exists in input and if so replaces it with the given char ${ucs:$j:1} (upper case) and stores it back into input.

```
input="${input/${input:$i:1}/${ucs:$j:1}}"
```

Share Follow

edited May 23 '17 at 12:10
Community Bot
1 1

answered Dec 23 '15 at 17:37
JaredTS486
426 5 15

This is wildly inefficient, looping 650 times in your example above, and taking 35 seconds to execute 1000 invocations on my machine. For an alternative that loops just 11 times and takes less than 5 seconds to execute 1000 invocations, see my alternative answer. – Dejay Clayton Jul 28 '18 at 17:20

1 Thanks, although that should be obvious just from looking at it. Perhaps the page faults are from the input size and the number of iterations you are executing. Nevertheless I like your solution. – JaredTS486 Aug 9 '18 at 19:24

3

To store the transformed string into a variable. Following worked for me - `$SOURCE_NAME` to `$TARGET_NAME`

```
TARGET_NAME="`echo $SOURCE_NAME | tr '[:upper:]' '[:lower:]'`"
```

Share Follow

edited Jul 17 '17 at 14:25

answered Jul 8 '16 at 9:20
nitinr708
1,305 2 19 27

2

For the Bash command line and depending on locale and international letters, this might work (assembled from the answers from others):

```
$ echo "ABCÆØÅ" | python -c "print(open(0).read().lower())"
abcæøå
$ echo "ABCÆØÅ" | sed 's/./\L&/g'
abcæøå
```



```
$ echo "ABCÆØÅ" | awk '{print tolower($1)}'
```

```
abcÆØÅ
$ echo "ABCÆØÅ" | perl -ne 'print lc'
abcÆØÅ
$ echo 'ABCÆØÅ' | dd conv=lcase 2> /dev/null
abcÆØÅ
```

Share Follow

answered Dec 2 at 13:29

Finn Årup Nielsen
5,110 1 23 35

1 Does `echo "ABCÆØÅ" | ruby -pe '$_.downcase!'` work correctly? – Fravadona Dec 2 at 14:48

Yes. The Ruby version also works with my setup. – Finn Årup Nielsen Dec 2 at 17:17

1

From the `bash` manpage:

> ${parameter^pattern}
>
> ${parameter^^pattern}
>
> ${parameter,pattern}
>
> ${parameter,,pattern}
>
> **Case modification**. This expansion modifies the case of alphabetic characters in *parameter*. The *pattern* is expanded to produce a pattern just as in pathname expansion. Each character in the expanded value of *parameter* is tested against *pattern*, and, if it matches the pattern, its case is converted. The pattern should not attempt to match more than one character. The **^** operator converts lowercase letters matching *pattern* to uppercase; the **,** operator converts matching uppercase letters to lowercase. The **^^** and **,,** expansions convert each matched character in the expanded value; the **^** and **,** expansions match and convert only the first character in the expanded value. If *pattern* is omitted, it is treated like a **?**, which matches every character. If *parameter* is **@** or *****, the case modification operation is applied to each positional parameter in turn, and the expansion is the resultant list. If *parameter* is an array variable subscripted with **@** or *****, the case modification operation is applied to each member of the array in turn, and the expansion is the resultant list.

0

transpose function (independently useful), returned the result in a variable (faster/safer), and added a BASH v4+ optimization:

```
pkg::transpose() { # <retvar> <string> <from> <to>
  local __r=$2 __m __p
  while [[ ${__r} =~ ([$3]) ]]; do
    __m="${BASH_REMATCH[1]}"; __p="${3%${__m}*}"
    __r="${__r//${__m}/${4:${#__p}:1}}"
  done
  printf -v "$1" "%s" "${__r}"
}
pkg::lowercase() { # <retvar> <string>
  if (( BASH_VERSINFO[0] >= 4 )); then
    printf -v "$1" "%s" "${2,,}"
  else
    pkg::transpose "$1" "$2" "ABCDEFGHIJKLMNOPQRSTUVWXYZ" \
                   "abcdefghijklmnopqrstuvwxyz"
  fi
}
pkg::uppercase() { # <retvar> <string>
  if (( BASH_VERSINFO[0] >= 4 )); then
    printf -v "$1" "%s" "${2^^}"
  else
    pkg::transpose "$1" "$2" "abcdefghijklmnopqrstuvwxyz" \
                   "ABCDEFGHIJKLMNOPQRSTUVWXYZ"
  fi
}
```

To keep things simple I didn't add any set -e support (or any error checking really)... but otherwise it generally follows shellguide and `pkg::transpose()` tries to avoid any likely variable name clashes for the `printf -v`

Share Follow

answered Sep 15 at 20:14

Scott Shambarger

31 2

---

use this command to do the same , it will convert upper case strings into lowercase :

0

```
sed 's/[A-Z]/[a-z]/g' <filename>
```

Share Follow

answered Dec 12 at 13:51

Piyush Pancholi

The reputation requirement helps protect this question from spam and non-answer activity.